

OPHR: Mastering Volatility Trading with Multi-Agent Deep RL

Google Colab Demo Notebook

Paper: NeurIPS 2025 | Chen, Cai, Qin, An (NTU / Skywork AI)

This notebook demonstrates the full OPHR pipeline:

1. Setup & Installation
2. Explore the codebase & data
3. Phase 1: Oracle experience collection
4. Phase 1: OP-Agent offline training
5. Phase 2: Iterative training (OP + HR agents)
6. Backtesting & evaluation
7. Visualization & metrics

Start coding or [generate](#) with AI.

1. Setup & Installation

```
# Clone the repository (includes sample data ~57MB)
!git clone https://github.com/Edwiczn/OPHR-MasteringVolatilityTradingwith
%cd OPHR-MasteringVolatilityTradingwithMultiAgentDeepReinforcementLearnin
fatal: destination path 'OPHR-MasteringVolatilityTradingwithMultiAgentDee
/content/OPHR-MasteringVolatilityTradingwithMultiAgentDeepReinforcementLe
```

```
# Install dependencies
!pip install -q torch numpy pandas matplotlib seaborn tqdm polars pyarrow
```

```
# Verify setup
import torch
import numpy as np
import os

print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
```

```
print(f"Device: {'cuda' if torch.cuda.is_available() else 'cpu'}")
print()

# Check sample data
data_dir = 'sample_data/BTC'
print(f"Sample data exists: {os.path.exists(data_dir)}")
print(f"Option chain files: {len(os.listdir(os.path.join(data_dir, 'opti
print(f"Perpetual data: {os.path.exists(os.path.join(data_dir, 'perp.par
print(f"Volatility tickers: {os.path.exists(os.path.join(data_dir, 'volt
print()
print("Setup complete!")
```

PyTorch version: 2.10.0+cu128

CUDA available: True

Device: cuda

Sample data exists: True

Option chain files: 91

Perpetual data: True

Volatility tickers: True

Setup complete!

✓ 2. Explore the Codebase & Data

```
# Show repository structure
!find . -name '*.py' -not -path './.git/*' | sort | head -50
```

```
./agents/hr_agent.py
./agents/__init__.py
./agents/op_agent.py
./agents/replay_buffer.py
./backtest/backtest.py
./backtest/__init__.py
./config.py
./env/agged.py
./env/base/account.py
./env/base/action.py
./env/base/call.py
./env/base_env.py
./env/base/__init__.py
./env/base/log.py
./env/base/option.py
./env/base/options_chain.py
./env/base/perpetual.py
./env/base/positions.py
./env/base/put.py
./env/base/tick.py
./env/base/trade.py
./env/base/volatility_tickers.py
./env/config.py
```

```

./env/data/data_handler.py
./env/data/data_handler_ray.py
./env/data/__init__.py
./env/data/utils.py
./env/evaluation/evaluation.py
./env/__init__.py
./env/rl_env.py
./evaluation/__init__.py
./evaluation/metrics.py
./evaluation/visualize.py
./hedgers/base_hedger.py
./hedgers/deep_hedger.py
./hedgers/delta_hedger.py
./hedgers/__init__.py
./hedgers/price_move_hedger.py
./oracle/__init__.py
./oracle/oracle_policy.py
./training/checkpoint_utils.py
./training/__init__.py
./training/phase1_hr_warmup.py
./training/phase1_op_offline.py
./training/phase1_oracle.py
./training/phase2_iterative.py

```

```

# Load and inspect configuration
# NOTE: config.py's TrainingConfig.from_yaml expects some keys that diff
# the shipped YAML schema. We monkey-patch it here so the notebook Just
# regardless of the repo version.
from config import ConfigManager, TrainingConfig, load_yaml

def _patched_training_from_yaml(cls, yaml_path):
    cfg = load_yaml(yaml_path)
    op = cfg['op_agent']
    hr = cfg['hr_agent']
    it = cfg['iterative_training']
    oracle = cfg['oracle']
    return cls(
        op_hidden_dims=op['hidden_dims'],
        op_activation=op.get('activation', 'relu'),
        op_learning_rate=op['learning_rate'],
        op_batch_size=op['batch_size'],
        op_replay_buffer_size=op['replay_buffer_size'],
        op_n_step=op['n_step'],
        op_gamma=op['gamma'],
        op_epsilon_start=op['epsilon_start'],
        op_epsilon_end=op['epsilon_end'],
        op_epsilon_decay=op.get('epsilon_decay', op.get('epsilon_decay_s
        op_update_frequency=op['update_frequency'],
        op_target_update_frequency=op['target_update_frequency'],
        oracle_episodes=op['oracle_episodes'],
        op_train_episodes=op['train_episodes'],
        op_eval_frequency=op['eval_frequency'],

```

```

        hr_hidden_dims=hr['hidden_dims'],
        hr_activation=hr.get('activation', 'relu'),
        hr_learning_rate=hr['learning_rate'],
        hr_batch_size=hr['batch_size'],
        hr_n_hr=hr['n_hr'],
        hr_train_episodes=hr['train_episodes'],
        num_iterations=it['num_iterations'],
        op_episodes_per_iter=it['op_episodes_per_iter'],
        hr_episodes_per_iter=it['hr_episodes_per_iter'],
        oracle_beta=oracle.get('beta', oracle.get('beta_thresholds', [0.
        oracle_lookforward_window=oracle['lookforward_window']
    )

```

```
TrainingConfig.from_yaml = classmethod(_patched_training_from_yaml)
```

```
config_manager = ConfigManager(data_root='sample_data', crypto='BTC')
```

```

print("=" * 50)
print("Environment Configuration")
print("=" * 50)
env_cfg = config_manager.env_config
print(f"Episode length: {env_cfg.episode_length} days")
print(f"Option interval: {env_cfg.option_interval} minutes")
print(f"Hedge interval: {env_cfg.hedge_interval} minutes")
print(f"Initial capital: {env_cfg.initial_capital} BTC")

```

```

print(f"\nData paths:")
for key, path in env_cfg.data_paths.items():
    print(f"    {key}: {path}")

```

```

print(f"\n{'=' * 50}")
print("Training Configuration")
print("=" * 50)
train_cfg = config_manager.training_config
print(f"OP-Agent hidden dims: {train_cfg.op_hidden_dims}")
print(f"OP-Agent learning rate: {train_cfg.op_learning_rate}")
print(f"OP-Agent n-step: {train_cfg.op_n_step}")
print(f"OP-Agent gamma: {train_cfg.op_gamma}")
print(f"OP-Agent batch size: {train_cfg.op_batch_size}")
print(f"HR-Agent n_hr: {train_cfg.hr_n_hr} hours")
print(f"Iterative iterations: {train_cfg.num_iterations}")
print(f"Oracle beta: {train_cfg.oracle_beta}")
print(f"Oracle lookforward: {train_cfg.oracle_lookforward_window}h")

```

```

=====
Environment Configuration
=====
Episode length: 14 days
Option interval: 180 minutes
Hedge interval: 60 minutes
Initial capital: 10.0 BTC

```

```
Data paths:
option_chain: sample_data/BTC/option_chain
perpetual: sample_data/BTC/perp.parquet
volatility_ticker: sample_data/BTC/volticker.parquet
valid_option_file: sample_data/BTC/valid_option_dict.pkl
```

```
=====
Training Configuration
=====
```

```
OP-Agent hidden dims: [1024, 1024]
OP-Agent learning rate: 0.0001
OP-Agent n-step: 12
OP-Agent gamma: 0.99
OP-Agent batch size: 512
HR-Agent n_hr: 24 hours
Iterative iterations: 5
Oracle beta: 0.1
Oracle lookforward: 168h
```

```
# Inspect the sample data
import pandas as pd

# Load perpetual futures data
perp_df = pd.read_parquet('sample_data/BTC/perp.parquet')
print("Perpetual Futures Data")
print(f"Shape: {perp_df.shape}")
print(f>Date range: {perp_df.index.min() if hasattr(perp_df.index, 'min')
print(f"Columns: {list(perp_df.columns[:15])}...")
print()
perp_df.head(3)
```

```
Perpetual Futures Data
Shape: (2184, 1187)
Date range: 0
Columns: ['timestamp', 'symbol', 'ask1_price', 'ask1_size', 'bid1_price',
```

	timestamp	symbol	ask1_price	ask1_size	bid1_price	bid1_size	ask
0	2024-01-01 00:00:00	BTC- PERPETUAL	42302.0	42910.0	42301.5	71190.0	
1	2024-01-01 01:00:00	BTC- PERPETUAL	42485.5	183740.0	42485.0	10.0	
2	2024-01-01 02:00:00	BTC- PERPETUAL	42625.5	38360.0	42625.0	91480.0	

3 rows × 1187 columns

```
# Inspect option chain data (one day)
option_files = sorted(os.listdir('sample_data/BTC/option_chain'))
```

```

print(f"Option chain files: {len(option_files)} days")
print(f"First: {option_files[0]}, Last: {option_files[-1]}")

sample_options = pd.read_parquet(f'sample_data/BTC/option_chain/{option_
print(f"\nSample option chain ({option_files[0]}):")
print(f"Shape: {sample_options.shape}")
print(f"Columns: {list(sample_options.columns[:15])}...")
sample_options.head(3)

```

Option chain files: 91 days

First: 2024-01-01.parquet, Last: 2024-03-31.parquet

Sample option chain (2024-01-01.parquet):

Shape: (4656, 23)

Columns: ['symbol', 'timestamp', 'exchange', 'type', 'strike_price', 'exp

	symbol	timestamp	exchange	type	strike_price	expiration	open_inte
0	BTC- 23FEB24- 49000-C	2024-01-01	deribit	call	49000	2024-02-23 08:00:00	
1	BTC- 29MAR24- 37000-C	2024-01-01	deribit	call	37000	2024-03-29 08:00:00	
2	BTC- 29MAR24- 60000-C	2024-01-01	deribit	call	60000	2024-03-29 08:00:00	2

3 rows × 23 columns

✓ 3. Explore Agent Architectures

Before training, let's look at the agent neural networks and understand what they do.

```

from agents.op_agent import OPAgent
from agents.hr_agent import HRAgent

# --- OP-Agent ---
print("=" * 60)
print("OP-Agent (Option Position Agent) -- The Trader")
print("=" * 60)

op_agent = OPAgent(
    state_dim=96,          # 48 volatility + 48 perpetual features
    hidden_dims=[1024, 1024],
    learning_rate=0.0001,
    gamma=0.99,

```

```

n_step=12                                # 12-hour look-ahead
)

print(f"\nQ-Network Architecture:")
print(op_agent.q_network)
total_params = sum(p.numel() for p in op_agent.q_network.parameters())
print(f"\nTotal parameters: {total_params:,}")
print(f"Device: {op_agent.device}")
print(f"Actions: 0=Long, 1=Neutral, 2=Short")

# Simulate action selection
fake_state = {
    'volatility_tickers': np.random.randn(48).astype(np.float32),
    'features': np.random.randn(48).astype(np.float32)
}
action = op_agent.select_action(fake_state, epsilon=0.0) # greedy
direction = op_agent.action_to_direction(action)

features = op_agent.extract_features(fake_state)
with torch.no_grad():
    features_tensor = torch.FloatTensor(features).unsqueeze(0).to(op_age
    q_values = op_agent.q_network(features_tensor)

print(f"\nExample (random weights, before training):")
print(f"  Q(Long):      {q_values[0,0].item():.4f}")
print(f"  Q(Neutral):    {q_values[0,1].item():.4f}")
print(f"  Q(Short):      {q_values[0,2].item():.4f}")
print(f"  Selected: {[ 'Long', 'Neutral', 'Short'][action]}")

```

```

=====
OP-Agent (Option Position Agent) -- The Trader
=====

```

```

Q-Network Architecture:
QNetwork(
  (network): Sequential(
    (0): Linear(in_features=96, out_features=1024, bias=True)
    (1): ReLU()
    (2): Linear(in_features=1024, out_features=1024, bias=True)
    (3): ReLU()
    (4): Linear(in_features=1024, out_features=3, bias=True)
  )
)

```

```

Total parameters: 1,152,003
Device: cuda
Actions: 0=Long, 1=Neutral, 2=Short

```

```

Example (random weights, before training):
  Q(Long):      -0.0580
  Q(Neutral):    0.0307
  Q(Short):      -0.1223
  Selected: Neutral

```

```

# --- HR-Agent ---
print("=" * 60)
print("HR-Agent (Hedger Routing Agent) -- The Risk Manager")
print("=" * 60)

hr_agent = HRAgent(
    state_dim=102,          # 96 market + 2 position + 4 Greeks
    hidden_dims=[1024, 1024],
    learning_rate=0.0001,
    n_hr=24                 # Decision every 24 hours
)

print(f"\nHR Q-Network Architecture:")
print(hr_agent.q_network)
total_params_hr = sum(p.numel() for p in hr_agent.q_network.parameters())
print(f"\nTotal parameters: {total_params_hr:,}")

print(f"\nHedger Pool ({hr_agent.num_hedgers} hedgers):")
for i, h in enumerate(hr_agent.hedgers):
    print(f"  Hedger {i}: {h}")

```

```

=====
HR-Agent (Hedger Routing Agent) -- The Risk Manager
=====

Created hedger pool with 8 hedgers:
  - Delta-based: 5
  - Price-based: 3
  - Deep hedgers: 0

HR Q-Network Architecture:
HRQNetwork(
  (network): Sequential(
    (0): Linear(in_features=102, out_features=1024, bias=True)
    (1): ReLU()
    (2): Linear(in_features=1024, out_features=1024, bias=True)
    (3): ReLU()
    (4): Linear(in_features=1024, out_features=8, bias=True)
  )
)

Total parameters: 1,163,272

Hedger Pool (8 hedgers):
  Hedger 0: DeltaThresholdHedger(threshold=0.05, ratio=1.0)
  Hedger 1: DeltaThresholdHedger(threshold=0.1, ratio=1.0)
  Hedger 2: DeltaThresholdHedger(threshold=0.2, ratio=1.0)
  Hedger 3: DeltaThresholdHedger(threshold=0.3, ratio=1.0)
  Hedger 4: DeltaThresholdHedger(threshold=3.0, ratio=0.0)
  Hedger 5: PriceMoveHedger(threshold=1.00%)
  Hedger 6: PriceMoveHedger(threshold=2.00%)
  Hedger 7: PriceMoveHedger(threshold=3.00%)

```


✓ 4. Phase 1: Oracle Experience Collection

The Oracle has access to future realized volatility (RV) and generates trading signals. We collect these experiences to initialize the OP-Agent's replay buffer.

Note: Using reduced episodes for demo speed. Paper uses 1000 episodes.

```
import datetime
import sys
import types

# Stub the missing `training.phase2_twin_env` module referenced by
# training/__init__.py (the repo ships a broken __init__). A purely
# synthetic stub with placeholder attributes is enough -- the notebook
# only uses collect_oracle_experience and train_iterative, neither of
# which actually needs phase2_twin_env.
if 'training.phase2_twin_env' not in sys.modules:
    _stub = types.ModuleType('training.phase2_twin_env')
    _stub.TwinEnvTrainer = type('TwinEnvTrainer', (), {})
    _stub.train_with_twin_env = lambda *a, **k: None
    sys.modules['training.phase2_twin_env'] = _stub

from training.phase1_oracle import collect_oracle_experience

config_manager = ConfigManager(data_root='sample_data', crypto='BTC')
data_paths = {
    'option_chain': 'sample_data/BTC/option_chain',
    'perpetual': 'sample_data/BTC/perp.parquet',
    'volatility_ticker': 'sample_data/BTC/volticker.parquet'
}

start_date = datetime.datetime(2024, 1, 1)
end_date = datetime.datetime(2024, 4, 1)

# Collect Oracle experience (reduced episodes for demo)
NUM_ORACLE_EPISODES = 5 # Paper uses 1000; use 5-10 for quick demo

print("Starting Oracle experience collection...")
print(f"Episodes: {NUM_ORACLE_EPISODES} (paper uses 1000)")
print()

op_agent, oracle_stats = collect_oracle_experience(
    crypto='BTC',
    start_date=start_date,
    end_date=end_date,
    num_episodes=NUM_ORACLE_EPISODES,
    config_manager=config_manager,
    data_paths=data_paths,
    save_checkpoint=False, # Don't save to disk for demo
```

```

        verbose=True
    )

```

```

Starting Oracle experience collection...
Episodes: 5 (paper uses 1000)

```

```

=====
Phase 1: Oracle Policy Initialization
=====

```

```

Oracle Settings:
  Beta: 0.1
  Lookforward window: 168h

```

```

OP-Agent initialized:
  State dim: 953
  Hidden dims: [1024, 1024]
  n-step: 12

```

```

Collecting Oracle experience (5 episodes)...
100%|██████████| 5/5 [01:17<00:00, 15.45s/it]
✓ Oracle experience collection completed

```

```

Statistics:
  Episodes: 5
  Avg return: 0.0318
  Avg length: 336.0 steps
  Replay buffer size: 1625

```

```

Signal distribution:
  Long: 0 (0.0%)
  Short: 1680 (100.0%)
  Neutral: 0 (0.0%)

```

```

# Visualize Oracle statistics
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Episode returns
axes[0].bar(range(len(oracle_stats['episode_returns'])), oracle_stats['e
axes[0].set_title('Oracle Episode Returns')
axes[0].set_xlabel('Episode')
axes[0].set_ylabel('Total Return (BTC)')
axes[0].axhline(y=0, color='red', linestyle='--', alpha=0.5)
axes[0].grid(True, alpha=0.3)

# Signal distribution
labels = ['Long', 'Short', 'Neutral']
counts = [oracle_stats['long_signals'], oracle_stats['short_signals'], o
colors = ['green', 'red', 'gray']
axes[1].bar(labels, counts, color=colors, alpha=0.7)

```

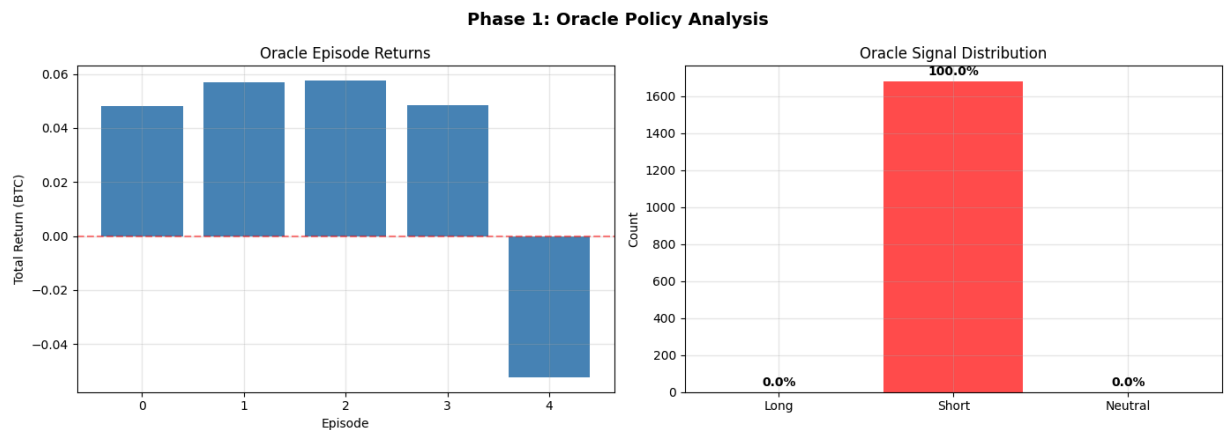
```

axes[1].set_title('Oracle Signal Distribution')
axes[1].set_ylabel('Count')
for i, v in enumerate(counts):
    total = sum(counts)
    axes[1].text(i, v + max(counts)*0.02, f'{v/total*100:.1f}%', ha='cen
axes[1].grid(True, alpha=0.3)

plt.suptitle('Phase 1: Oracle Policy Analysis', fontsize=14, fontweight=
plt.tight_layout()
plt.show()

print(f"\nReplay buffer size: {len(op_agent.replay_buffer)} transitions"

```



Replay buffer size: 1625 transitions

✓ 5. Phase 1: OP-Agent Offline Training

Train the OP-Agent on the Oracle replay buffer using n-step Double DQN.

```

# Offline training on Oracle experience
NUM_UPDATES = 100 # Paper does many more; use 100 for quick demo

print(f"Training OP-Agent offline ({NUM_UPDATES} updates)...")
print(f"Replay buffer has {len(op_agent.replay_buffer)} transitions")
print(f"Batch size: {op_agent.batch_size}")

```

```
print()

losses = []
for i in range(NUM_UPDATES):
    loss = op_agent.update()
    losses.append(loss)

    if (i + 1) % 10 == 0:
        op_agent.update_target_network()

    if (i + 1) % 25 == 0:
        print(f"  Update {i+1}/{NUM_UPDATES} | Loss: {loss:.6f}")

# Plot training loss
if any(l > 0 for l in losses):
    plt.figure(figsize=(10, 4))
    plt.plot(losses, alpha=0.6, color='blue', linewidth=0.8)
    # Moving average
    window = min(20, len(losses))
    if window > 1:
        ma = np.convolve(losses, np.ones(window)/window, mode='valid')
        plt.plot(range(window-1, len(losses)), ma, color='red', linewidth=0.8)
    plt.title('OP-Agent Offline Training Loss (n-step Double DQN)')
    plt.xlabel('Update Step')
    plt.ylabel('MSE Loss')
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.show()
else:
    print("Not enough data in buffer for training (need more Oracle epis
```

Training OP-Agent offline (100 updates)...

Replay buffer has 1625 transitions

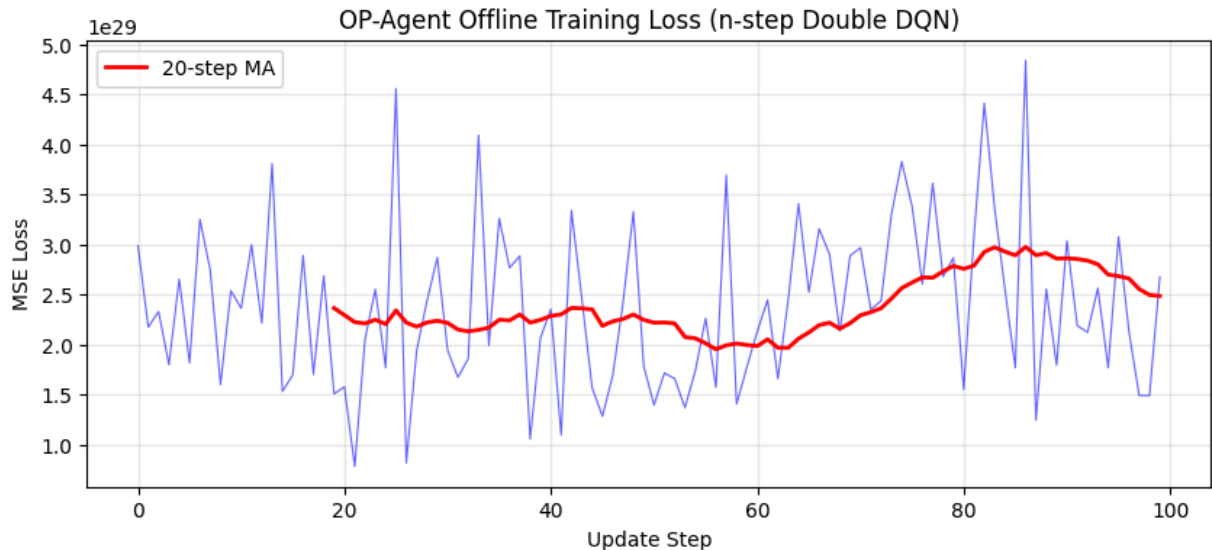
Batch size: 512

Update 25/100 | Loss: 176546048751400315885642055680.000000

Update 50/100 | Loss: 178182764305965140480966524928.000000

Update 75/100 | Loss: 382647689267147908091530444800.000000

Update 100/100 | Loss: 267231994615721616268272336896.000000



✓ 6. Phase 2: Iterative Training (OP + HR Agents)

Alternating training:

- Train OP-Agent with HR-Agent frozen
- Train HR-Agent with OP-Agent frozen (using twin environment)

Demo mode: We override the YAML config to **1 iteration × (2 OP + 1 HR) = 3 episodes** so this fits in a few minutes on Colab CPU. The paper uses 5 iterations × (200 OP + 50 HR) = 1,250 episodes.

```
from training.phase2_iterative import train_iterative

# Initialize HR-Agent
# First, get state dimensions from environment
from env.data.data_handler import DataHandler
from env.base_env import BaseEnv
```

```

from env.rl_env import RLEnv

env_cfg = config_manager.env_config
data_handler = DataHandler(
    start_date, end_date, 'BTC',
    data_paths['option_chain'],
    data_paths['perpetual'],
    volatility_ticker_path=data_paths.get('volatility_ticker')
)
time_config = {
    'episode_length': env_cfg.episode_length,
    'option_interval': env_cfg.option_interval,
    'include_volatility_tickers': True
}
base_env = BaseEnv(data_handler, time_config, crypto='BTC')
env = RLEnv(base_env)
state, info = env.reset(start_date, end_date)

feat = np.concatenate([
    state.get('volatility_tickers', np.zeros(48)),
    state.get('features', np.zeros(48))
])
state_dim = len(feat)
hr_state_dim = state_dim + 2 + 4 # + position features + Greeks

print(f"OP state dim: {state_dim}")
print(f"HR state dim: {hr_state_dim}")

train_cfg = config_manager.training_config

hr_agent = HRAgent(
    state_dim=hr_state_dim,
    hidden_dims=train_cfg.hr_hidden_dims,
    learning_rate=train_cfg.hr_learning_rate,
    n_hr=train_cfg.hr_n_hr
)

```

```

OP state dim: 953
HR state dim: 959
Created hedger pool with 8 hedgers:
- Delta-based: 5
- Price-based: 3
- Deep hedgers: 0

```

```

# Run iterative training (HEAVILY REDUCED for 10-15 min total demo)
# Paper uses 5 iterations x (200 OP + 50 HR) episodes = 1,250 episodes.
# For a demo that finishes on Colab CPU in minutes, we shrink to
# 1 iteration x (2 OP + 1 HR) = 3 episodes. This proves the pipeline run
# end-to-end; it will NOT reproduce paper-level performance.
train_cfg = config_manager.training_config
train_cfg.num_iterations = 2

```

```

train_cfg.op_episodes_per_iter = 10
train_cfg.hr_episodes_per_iter = 5

print("Starting Phase 2: Iterative Training (DEMO MODE)")
print(f"  Iterations: {train_cfg.num_iterations}")
print(f"  OP episodes/iter: {train_cfg.op_episodes_per_iter}")
print(f"  HR episodes/iter: {train_cfg.hr_episodes_per_iter}")
print(f"  Total episodes: {train_cfg.num_iterations * (train_cfg.op_episodes_per_iter + train_cfg.hr_episodes_per_iter)}")
print("  (Paper uses 1,250 total episodes)")
print()

op_agent_trained, hr_agent_trained = train_iterative(
    crypto='BTC',
    start_date=start_date,
    end_date=end_date,
    config_manager=config_manager,
    data_paths=data_paths,
    op_agent=op_agent,
    hr_agent=hr_agent,
    save_checkpoint=False,
    verbose=True
)

```

```

Starting Phase 2: Iterative Training (DEMO MODE)
  Iterations: 2
  OP episodes/iter: 10
  HR episodes/iter: 5
  Total episodes: 30
  (Paper uses 1,250 total episodes)

```

```

=====
Phase 2: Iterative Training with Twin Environment
=====

```

This implements Algorithm 2 from the paper:

- Alternating training of OP-Agent and HR-Agent
- HR-Agent uses twin environment for relative reward
- Baseline hedger: DeltaThresholdHedger(threshold=0.1)

```

=====
Iteration 1/2
=====

```

```

[OP-Agent Training] 10 episodes
100%|██████████| 10/10 [05:31<00:00, 33.13s/it]
  OP avg reward: -0.3759, avg final value: 9.6241

```

```

[HR-Agent Training] 5 episodes (Twin Env)
100%|██████████| 5/5 [01:55<00:00, 23.00s/it]
  HR avg reward: 0.0000, avg decisions: 0.0

```

✓ Checkpoints saved for iteration 1

```

=====

```

Iteration 2/2

=====

[OP-Agent Training] 10 episodes

100%|██████████| 10/10 [06:33<00:00, 39.37s/it]

OP avg reward: -0.5240, avg final value: 9.4760

[HR-Agent Training] 5 episodes (Twin Env)

100%|██████████| 5/5 [02:23<00:00, 28.68s/it] HR avg reward: 0.0000, avg

✓ Checkpoints saved for iteration 2

=====

✓ Phase 2: Iterative Training Completed!

=====

✓ 7. Backtesting & Evaluation

Run the trained agents on the test data and compare against baseline.

```
# Stub the missing `backtest.compare` module referenced by backtest/__in
import sys, types
if 'backtest.compare' not in sys.modules:
    _stub = types.ModuleType('backtest.compare')
    _stub.compare_strategies = lambda *a, **k: None
    _stub.quick_comparison = lambda *a, **k: None
    sys.modules['backtest.compare'] = _stub

from backtest.backtest import BacktestRunner
from decimal import Decimal

def _safe_run_episode(self):
    """Patched run_episode that tolerates running past end-of-data.
    The env raises a polars ColumnNotFoundError when it queries beyond t
    last timestamp in sample_data. We trap it and return the partial res
    """
    state, info = self.env.reset(self.start_date, self.end_date)
    done = False
    steps = 0

    results = {
        'timestamps': [], 'net_values': [], 'rewards': [], 'actions': []
        'hedger_selections': [], 'option_positions': [], 'perp_positions
        'greeks': {'delta': [], 'gamma': [], 'theta': [], 'vega': []},
        'underlying_prices': []
    }
```



```

if self.mode in ['full', 'op_only'] and self.hr_agent is not None:
    self.hr_agent.reset_decision_counter()

from tqdm import tqdm as _tqdm
from backtest.backtest import _build_option_action_from_direction, _
import numpy as _np

with _tqdm(total=None, disable=not self.verbose, desc="Backtest") as
    while not done:
        try:
            timestamp = state['timestamp']
            greeks = state['greeks']
            position = info['position']
            net_value = info['account'].net_value

            options_chain = state.get('options_chain', {})
            underlying_price = None
            if options_chain:
                first_option = next(iter(options_chain.values()))
                underlying_price = float(first_option.underlying_pri

            if self.mode == 'baseline':
                direction = 0
                option_action = _build_option_action_from_direction(
                hedge_action = _build_hedge_action(state, greeks, se
                hedger_name = 'baseline'
            elif self.mode == 'op_only':
                action_idx = self.op_agent.select_action(state, epsi
                direction = self.op_agent.action_to_direction(action
                option_action = _build_option_action_from_direction(
                hedge_action = _build_hedge_action(state, greeks, se
                hedger_name = 'baseline'
            elif self.mode == 'full':
                action_idx = self.op_agent.select_action(state, epsi
                direction = self.op_agent.action_to_direction(action
                option_action = _build_option_action_from_direction(
                _ = self.hr_agent.step(state, position, greeks)
                current_hedger = self.hr_agent.hedgers[self.hr_agent
                hedge_action = _build_hedge_action(state, greeks, cu
                hedger_name = f"hedger_{self.hr_agent.current_hedger

            next_state, reward, done, next_info = self.env.step(opti

            results['timestamps'].append(timestamp)
            results['net_values'].append(float(net_value))
            results['rewards'].append(float(reward))
            results['actions'].append(direction)
            results['hedger_selections'].append(hedger_name)
            results['option_positions'].append(len(position.option_p
            results['perp_positions'].append(float(position.perpetua
            results['greeks']['delta'].append(float(greeks[0]))

```

```

results['greeks']['gamma'].append(float(greeks[1]))
results['greeks']['theta'].append(float(greeks[2]))
results['greeks']['vega'].append(float(greeks[3]))
results['underlying_prices'].append(underlying_price if

state = next_state
info = next_info
steps += 1
pbar.update(1)
except Exception as e:
    if self.verbose:
        print(f"\n [info] Backtest stopped early at step {s
break

results['total_steps'] = steps
results['initial_value'] = results['net_values'][0] if results['net_
results['final_value'] = results['net_values'][-1] if results['net_v
results['total_return'] = results['final_value'] - results['initial_
results['return_pct'] = (results['total_return'] / results['initial_
return results

```

```
BacktestRunner.run_episode = _safe_run_episode
```

```

# --- Run OPHR (full: OP + HR agents) ---
print("Running backtest: OPHR (OP-Agent + HR-Agent)...")
runner_full = BacktestRunner(
    crypto='BTC',
    start_date=start_date,
    end_date=end_date,
    config_manager=config_manager,
    data_paths=data_paths,
    op_agent=op_agent_trained,
    hr_agent=hr_agent_trained,
    mode='full',
    verbose=True
)
results_full = runner_full.run_episode()
metrics_full = runner_full.calculate_metrics(results_full)
print(f"\nOPHR return: {metrics_full['return_pct']:.2f}% over {results_f

```

```
Running backtest: OPHR (OP-Agent + HR-Agent)...
```

```
=====
```

```
Backtest Runner Initialized
```

```
=====
```

```
Mode: full
```

```
Crypto: BTC
```

```
Period: 2024-01-01 to 2024-04-01
```

```
OP-Agent: Loaded
```

```
HR-Agent: Loaded
```

```
Baseline Hedger: DeltaThresholdHedger
```

```
Backtest: 2175it [03:04, 11.81it/s]
```

[info] Backtest stopped early at step 2175: ColumnNotFoundError

OPHR return: -33.22% over 2175 steps

```
# --- Run OP-only (OP-Agent + baseline hedger) ---
print("\nRunning backtest: OP-only (OP-Agent + baseline hedger)...")
runner_op = BacktestRunner(
    crypto='BTC',
    start_date=start_date,
    end_date=end_date,
    config_manager=config_manager,
    data_paths=data_paths,
    op_agent=op_agent_trained,
    hr_agent=hr_agent_trained,
    mode='op_only',
    verbose=False
)
results_op = runner_op.run_episode()
metrics_op = runner_op.calculate_metrics(results_op)
print(f"OP-only return: {metrics_op['return_pct']:.2f}%")
```

Running backtest: OP-only (OP-Agent + baseline hedger)...
OP-only return: -33.22%

```
# --- Compare results ---
print("\n" + "=" * 60)
print("PERFORMANCE COMPARISON")
print("=" * 60)

comparison = {
    'Metric': ['Total Return (%)', 'Final Value (BTC)', 'Sharpe Ratio',
              'Max Drawdown (%)', 'Calmar Ratio', 'Sortino Ratio',
              'Long %', 'Short %', 'Neutral %'],
    'OPHR (Full)': [
        f"{metrics_full['return_pct']:.2f}",
        f"{metrics_full['final_value']:.4f}",
        f"{metrics_full.get('sharpe_ratio', 0):.4f}",
        f"{metrics_full.get('max_drawdown', 0):.2f}",
        f"{metrics_full.get('calmar_ratio', 0):.4f}",
        f"{metrics_full.get('sortino_ratio', 0):.4f}",
        f"{metrics_full.get('long_ratio', 0):.1f}",
        f"{metrics_full.get('short_ratio', 0):.1f}",
        f"{metrics_full.get('neutral_ratio', 0):.1f}"
    ],
    'OP-Only': [
        f"{metrics_op['return_pct']:.2f}",
        f"{metrics_op['final_value']:.4f}",
        f"{metrics_op.get('sharpe_ratio', 0):.4f}",
        f"{metrics_op.get('max_drawdown', 0):.2f}",

```

```

        f"{metrics_op.get('calmar_ratio', 0):.4f}",
        f"{metrics_op.get('sortino_ratio', 0):.4f}",
        f"{metrics_op.get('long_ratio', 0):.1f}",
        f"{metrics_op.get('short_ratio', 0):.1f}",
        f"{metrics_op.get('neutral_ratio', 0):.1f}"
    ]
}

df_comparison = pd.DataFrame(comparison)
print(df_comparison.to_string(index=False))

```

✓ 8. Visualization

```

# Portfolio Value (PnL Curve)
fig, axes = plt.subplots(3, 2, figsize=(16, 14))
fig.suptitle('OPHR Backtest Results (BTC)', fontsize=16, fontweight='bold')

# 1. PnL Curve comparison
ax = axes[0, 0]
ax.plot(results_full['net_values'], label='OPHR (Full)', color='blue', lw=1)
ax.plot(results_op['net_values'], label='OP-Only', color='orange', lw=1)
ax.set_title('Portfolio Value Over Time')
ax.set_xlabel('Time (hours)')
ax.set_ylabel('Portfolio Value (BTC)')
ax.legend()
ax.grid(True, alpha=0.3)

# 2. Underlying Price
ax = axes[0, 1]
ax.plot(results_full['underlying_prices'], color='black', linewidth=1)
# Color background by position
actions = results_full['actions']
for i in range(len(actions)-1):
    if actions[i] == 1:
        ax.axvspan(i, i+1, alpha=0.1, color='green')
    elif actions[i] == -1:
        ax.axvspan(i, i+1, alpha=0.1, color='red')
ax.set_title('BTC Price (green=long, red=short)')
ax.set_xlabel('Time (hours)')
ax.set_ylabel('Price (USD)')
ax.grid(True, alpha=0.3)

# 3. Delta Exposure
ax = axes[1, 0]
ax.plot(results_full['greeks']['delta'], color='green', linewidth=0.8)
ax.axhline(y=0, color='black', linestyle='--', alpha=0.3)
ax.set_title('Delta Exposure (should stay near 0 = good hedging)')
ax.set_xlabel('Time (hours)')

```

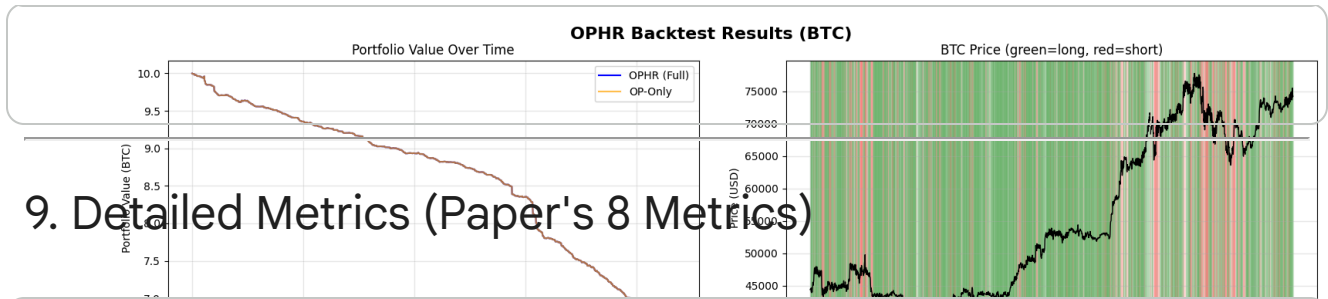
```
ax.set_ylabel('Delta')
ax.grid(True, alpha=0.3)

# 4. Gamma Exposure
ax = axes[1, 1]
ax.plot(results_full['greeks']['gamma'], color='red', linewidth=0.8)
ax.axhline(y=0, color='black', linestyle='--', alpha=0.3)
ax.set_title('Gamma Exposure (positive=long vol, negative=short vol)')
ax.set_xlabel('Time (hours)')
ax.set_ylabel('Gamma')
ax.grid(True, alpha=0.3)

# 5. Position Actions
ax = axes[2, 0]
colors_map = {1: 'green', 0: 'gray', -1: 'red'}
colors_list = [colors_map[a] for a in actions]
ax.scatter(range(len(actions)), actions, c=colors_list, s=2, alpha=0.5)
ax.set_title('OP-Agent Actions (1=Long, 0=Neutral, -1=Short)')
ax.set_xlabel('Time (hours)')
ax.set_ylabel('Action')
ax.set_yticks([-1, 0, 1])
ax.set_yticklabels(['Short', 'Neutral', 'Long'])
ax.grid(True, alpha=0.3)

# 6. Hedger Selection Distribution
ax = axes[2, 1]
hedger_counts = pd.Series(results_full['hedger_selections']).value_count
hedger_counts.plot(kind='bar', ax=ax, color='steelblue', alpha=0.7)
ax.set_title('HR-Agent Hedger Selection Frequency')
ax.set_xlabel('Hedger')
ax.set_ylabel('Count')
ax.grid(True, alpha=0.3)
plt.xticks(rotation=45)

plt.tight_layout()
plt.savefig('ophr_backtest_results.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure saved to ophr_backtest_results.png")
```

9. Detailed Metrics (Paper's 8 Metrics)

```
from evaluation.metrics import calculate_all_metrics, print_metrics
```

```
print("=" * 60)
print("OPHR (Full) -- Detailed Performance Metrics")
print("=" * 60)
```

```
detailed_metrics = calculate_all_metrics(
    pnl_history=results_full['net_values'],
    direction_history=results_full['actions'],
    hourly_sampling=24, # Sample daily for metric calculation
    annual_days=365,
    risk_free_rate=0.0
)
```

```
print_metrics(detailed_metrics)
```

```
print(f"\n{'=' * 60}")
print("OP-Only -- Detailed Performance Metrics")
print("=" * 60)
```

```
detailed_metrics_op = calculate_all_metrics(
    pnl_history=results_op['net_values'],
    direction_history=results_op['actions'],
    hourly_sampling=24,
    annual_days=365,
    risk_free_rate=0.0
)
```

```
print_metrics(detailed_metrics_op)
```

```
# Trade analysis
```

```
from evaluation.metrics import extract_trades
```

```
trade_returns, trade_directions = extract_trades(
    results_full['actions'],
    np.array(results_full['net_values'])
)
```

```
if len(trade_returns) > 0:
    long_trades = trade_returns[trade_directions == 1]
    short_trades = trade_returns[trade_directions == -1]
```

```

print(f"\nTrade Analysis:")
print(f"  Total trades: {len(trade_returns)}")
print(f"  Long trades: {len(long_trades)} (WR: {np.sum(long_trades)}")
print(f"  Short trades: {len(short_trades)} (WR: {np.sum(short_trades)}")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Trade return distribution
if len(long_trades) > 0:
    axes[0].hist(long_trades, bins=20, alpha=0.6, label=f'Long ({len(long_trades)})')
if len(short_trades) > 0:
    axes[0].hist(short_trades, bins=20, alpha=0.6, label=f'Short ({len(short_trades)})')
axes[0].set_title('Trade Return Distribution')
axes[0].set_xlabel('Return (BTC)')
axes[0].set_ylabel('Frequency')
axes[0].legend()
axes[0].grid(True, alpha=0.3)
axes[0].axvline(x=0, color='black', linestyle='--', alpha=0.5)

# Cumulative trade PnL
cumulative_pnl = np.cumsum(trade_returns)
axes[1].plot(cumulative_pnl, linewidth=2, color='blue')
axes[1].set_title('Cumulative Trade PnL')
axes[1].set_xlabel('Trade Number')
axes[1].set_ylabel('Cumulative PnL (BTC)')
axes[1].grid(True, alpha=0.3)
axes[1].axhline(y=0, color='red', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
else:

```

10. Summary

What We Demonstrated:

1. **Oracle Initialization** -- Collected trading experience using a teacher policy with future information